Numbers you can *defend.*

Drop a messy CSV. Approve the business definition. Proofsheet executes the analysis in a disposable sandbox, then proves every number twice.

Definition locked **2 executions agree** **0 unverified claims**



THE GAP

An AI chart is not *evidence.*

A model can invent a plausible figure inside a chart. The analyst still has to clean the export, re-check the math, and defend the number in the room.

80%

of analysis time

Spent cleaning exports, not analyzing them.

1

wrong number

Enough to lose trust in the whole report.

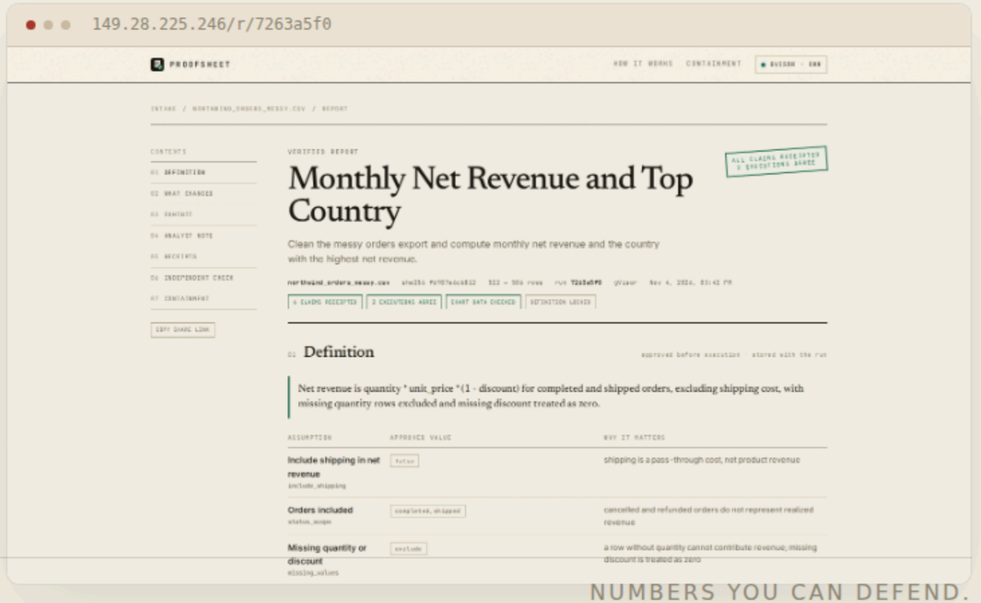
0

receipts

A normal AI data tool shows a chart, not the run.

> “A fabricated number in a chart looks exactly like a real one.”

The failure is not that the model cannot compute. It is that the output carries no definition, no execution, no independent check, and no receipt.



NUMBERS YOU CAN DEFEND.

THE PRODUCT

Plan. Lock. Execute. *Prove.*

A single Rust control plane profiles the file, asks Vultr for a plan and code, dispatches execution to a disposable gVisor sandbox, and verifies the report against the ledger.

01

Profile

Duplicates, five date formats, currency text, percent strings, nulls and casing are mapped in Rust before a model sees the file.

02

Lock

The user approves shipping, status scope, missing values, discount semantics and date precedence. The definition is stored with the run.

03

Execute

The generated script runs in gVisor with no network, no secrets, a read-only root, and hard time, memory and process caps.

04

Prove

The note cites placeholders, never figures. Rust resolves them from the ledger, audits the chart, and runs a second implementation.

The model may plan and write code. It cannot author a number.

ONE CONTROL PLANE · ALL INFERENCE THROUGH VULTR

DEFINITION LOCK

Same definition. *Every time.*

Before any code runs, the user approves the business rules. The definition is stored with the run, displayed on the report, and reused on replay.

Shipping in net revenue	`false`	A pass-through cost, not product revenue.
Orders included	`completed, shipped`	Cancelled and refunded orders are not realized revenue.
Missing values	`exclude`	A row without a metric input cannot be counted.
Ambiguous dates	`day-first + precedence`	Explicit format order; one parser for both executions.
Duplicate rows	`drop exact`	Exact duplicates would double-count revenue.
Missing discount	`zero`	An empty discount means no discount was applied.
Discount text	`percent → fraction`	10% becomes 0.10; 0.10 is already 0.10.

THE GATE

The model cannot change these after approval.

A wrong definition is a wrong answer, even if every number is receipted. The lock makes the choice explicit before the code exists.

USED BY THE PRIMARY SCRIPT
USED BY THE CHECKER
USED ON EVERY REPLAY

CONTAINMENT

The blast radius is *zero.*

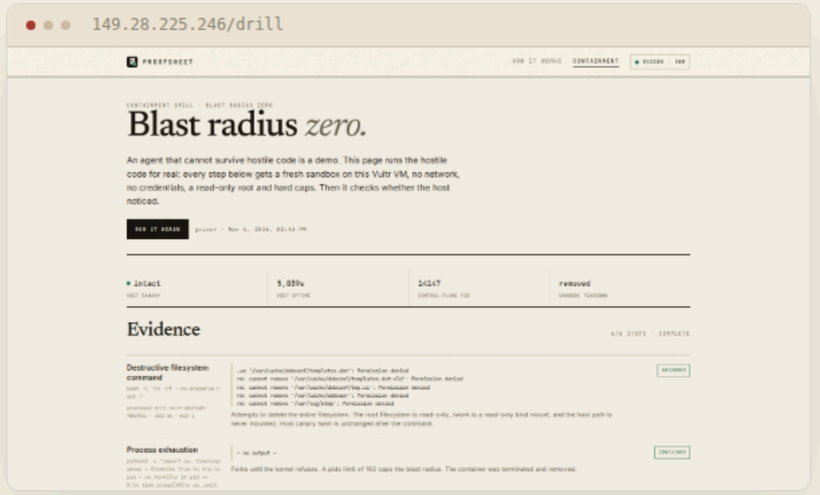
Execution never touches the app process or the host filesystem. Every task gets a fresh gVisor sandbox; every sandbox is force-removed afterwards.

6 / 6
Drill steps
Run live on the production VM.

intact
Host canary
Byte-identical before and after.

0
Secrets inside
No Vultr or inference key in the sandbox.

Destructive filesystem	`rm -rf --no-preserve-root /`	ABSORBED
Process exhaustion	`fork until the kernel refuses`	CONTAINED
Runaway CPU loop	`while true: pass`	CONTAINED
Credential scan	`env + common secret paths`	BLOCKED
Outbound network	`connect 1.1.1.1:80`	BLOCKED
Host filesystem	`read the host canary`	BLOCKED



gVisor user-space kernel · no network · read-only root · 1 CPU / 1 GB / 90 s · pids 192 · container removed.

RECEIPTS

Every number has a *receipt.*

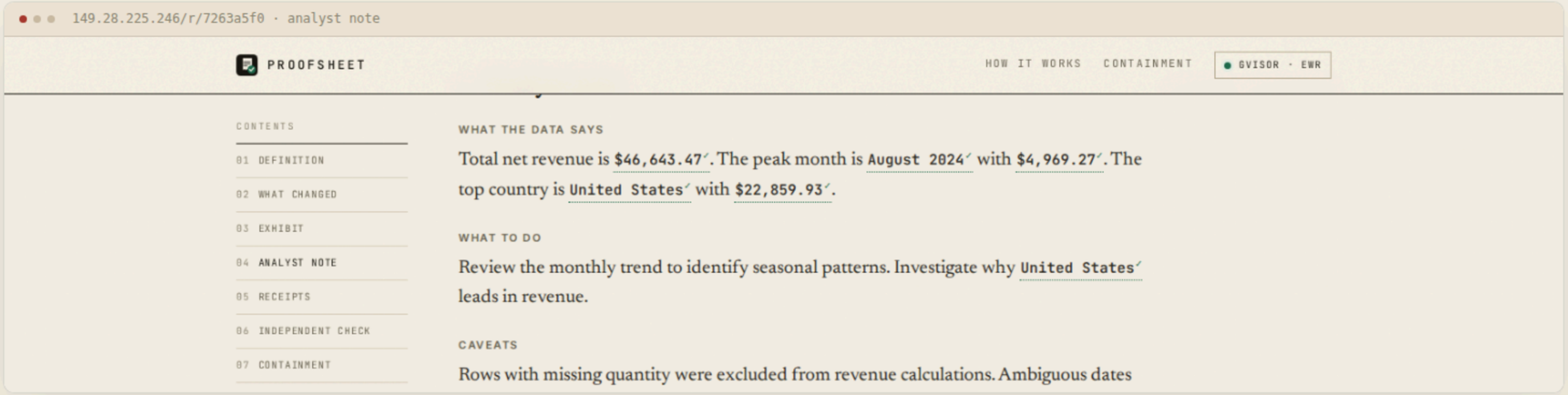
The note cites placeholders, never figures. Rust resolves each one from the execution ledger, audits the chart series against the same numbers, and opens the exact run.

6 / 6
Claims receipted
Every figure and name has a receipt.

0
Unverified
No literal digits slipped through.

8 / 8
Chart checks
Series, top category, peak, assumptions.

333
Rows used
The rest excluded by the definition.



INDEPENDENT EXECUTION

Two executions, *one answer.*

A second model, which never sees the analyst script, recomputes the same metrics from the approved definition in a separate sandbox. Both import the same deterministic parsing contract.

5 / 5

Metrics agree

Including names and months.

0.0%

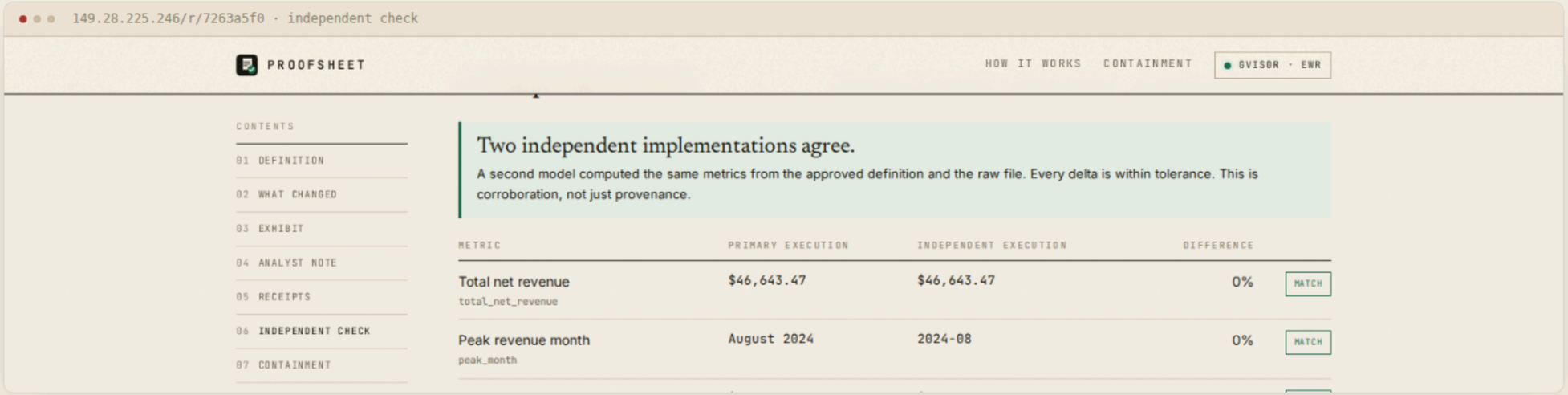
Difference

Every delta inside the materiality threshold.

2

Sandboxes

Primary and checker, both destroyed.



During development the checker caught two real ambiguities — discount semantics and date precedence. Both became locked assumptions. The disagreement was the feature working.

ADVERSARIAL CHECK

Try to make it *lie.*

Inject a fake forty-seven percent and an unknown metric into the note. The same verifier that stamped the report runs again — and the stamp disappears.

2

Flags raised

A literal number and an unknown metric.

0

Stamp

The report cannot be verified once a claim is unreceipted.

10s

To reproduce

A live, deterministic check on the real ledger.



THE RETENTION LOOP

The second month is *one click.*

A replay re-executes the saved definition and script on a new export with no model in the loop, then computes the period comparison from two ledgers in Rust.

506 → 506

Rows

Same scope, new period.

+3.0%

Net revenue

$46,643.47 → $48,059.55.

0

Model calls

The saved pipeline replays deterministically.



WHAT IS REAL · WHAT IS NEXT

Audit-ready reporting for the *Monday export.*

REAL TODAY	NOT YET
Definition lock stored with every run	Authentication and team workspaces
Rust profiler and residual-dirt audit	Billing and scheduled deliveries
gVisor execution with no network or secrets	One microVM per task at production scale
Chart audit against the execution ledger	Write-back to a warehouse
Independent second implementation	
Live containment drill and host canary	
Assurance bundle with integrity hash	
Deterministic replay + period comparison	

15/15 GOLDEN CHECKS · 6/6 VERIFIER · 3/3 FIXTURES
gVISOR · VULTR SERVERLESS INFERENCE · EWR